LAB SESSION 08

OEL (Open Ended Lab)

OBJECTIVE:

To interface analog voltage sensor ZMPT101B for measurement of phase voltage and display its true RMS (Root Mean Square) value on LCD (Liquid Crystal Display) screen.

DELIVERABLES:

- Report
- C-Code
- Complete hardware setup (ZMPT101B module and LCD interfaced with Microprocessor)

INTRODUCTION:

The ZMPT101B is a precision voltage transformer module widely used for AC voltage sensing in embedded systems. It converts the high AC mains voltage (220V, 50 Hz) down to a safe, measurable analog signal (typically in the range of 0–5V) that can be read by a microcontroller's ADC (Analog-to-Digital Converter). The module incorporates a built-in operational amplifier for signal conditioning, ensuring accurate and stable readings.

In this lab, the ZMPT101B is interfaced with an Arduino UNO microcontroller. The Arduino samples the analog output of the ZMPT101B and computes both the average and the true RMS voltage. The RMS value is particularly important in AC systems because it represents the equivalent DC value that would deliver the same power to a resistive load.

COMPONENTS USED:

- Arduino UNO (ATmega328P microcontroller)
- ZMPT101B Voltage Sensor Module
- Breadboard and connecting wires
- 9 Volt Battery
- AC Power Source (230V, 50Hz mains supply)
- Oscilloscope (for waveform verification)
- USB Cable (for programming and serial communication)
- SSD1306 I2C OLED (0.96 inch)
- Tera Term Serial Monitor

THEORY:

AC voltage measurement requires computing the Root Mean Square (RMS) value of the signal. For a pure sinusoidal signal, the RMS value is given by:

$$V_{RMS} = \frac{V_{peak}}{\sqrt{2}} \approx 0.7071 \times V_{peak}$$

However, mains voltage is not always a perfect sine wave. Therefore, a true RMS calculation is performed by sampling the waveform over one full cycle, squaring the samples, computing their mean, and taking the square root:

$$V_{RMS} = \sqrt{\left(\frac{1}{N}\right) \times \Sigma\, v_i^2}$$

The Arduino's 10-bit ADC converts the analog voltage (0–5V) to a digital value (0–1023). The ADC readings are centered around 512 (midpoint), and the offset is removed before RMS computation. A calibration factor is then applied to map the measured small-signal RMS to the actual mains voltage.

HARDWARE SETUP:

The ZMPT101B AC voltage sensor module's analog output (OUT pin) is connected to the A0 pin of the Arduino UNO. The module is powered from the Arduino's 5V and GND rails. The AC mains (220V, 50Hz) is connected to the primary input terminals of the ZMPT101B module via alligator clips.

The SSD1306 OLED display (128×64, I2C) is connected to the Arduino via the I2C bus SDA to A4 and SCL to A5 and powered from the 5V rail. The OLED displays a startup screen showing project title, lecturer name (Sir HassanUlHaq), and group leader (Muhammad Taha EE-319), followed by live Vavg and Vrms readings.

The Arduino UNO is connected to a PC via USB for programming (hex file flashed via AVRDUDE) and serial data monitoring through Tera Term at 9600 baud. Tera Term displays a header on startup followed by continuously updating Vavg and Vrms values on new lines every 500ms.

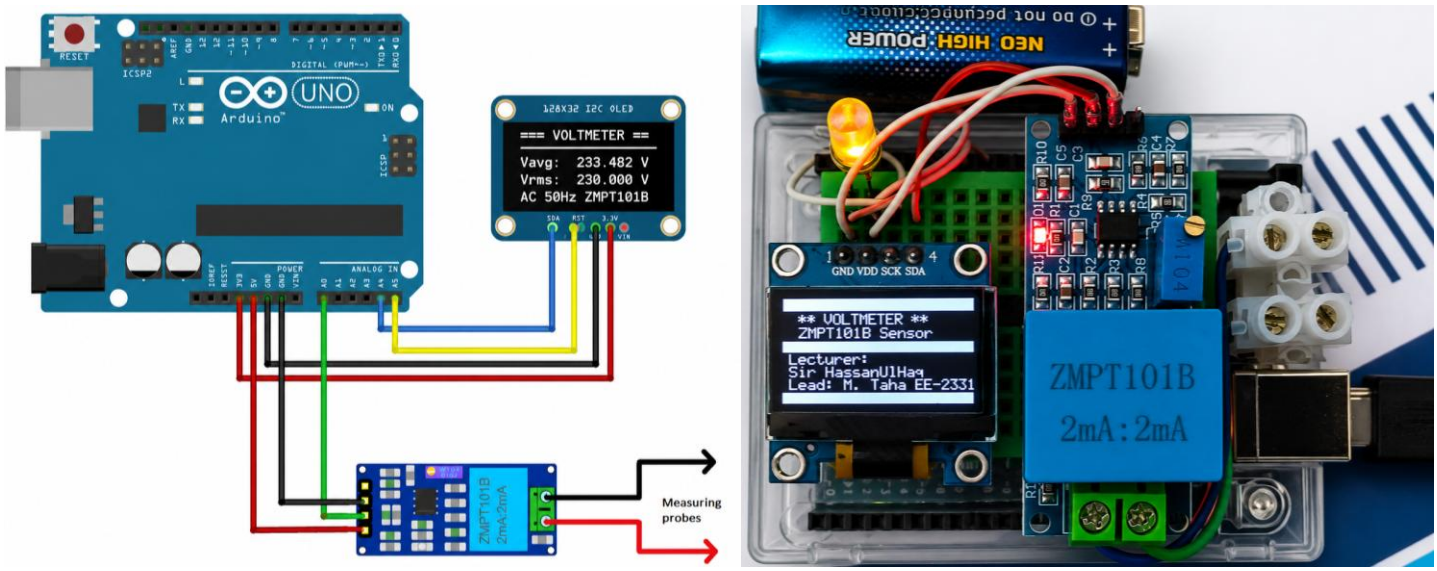

Figure 1: Hardware Setup – Arduino UNO with ZMPT101B Module

C-CODE :

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/pgmspace.h>
#include <util/delay.h>
#include <stdint.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
#define BAUD      9600UL
#define UBRR_VAL  (F_CPU / 16UL / BAUD -
1)
static void i2c_init(void)
{
    TWBR = (uint8_t)(((F_CPU / 400000UL) -
16UL) / 2UL);
    TWSR &= ~((1 << TWPS1) | (1 <<
TWPS0));
    TWCR  = (1 << TWEN);
}

static void i2c_start(void)
{
    TWCR = (1 << TWINT) | (1 << TWSTA) |
(1 << TWEN);
    while (!(TWCR & (1 << TWINT)));
}

static void i2c_stop(void)
{
    TWCR = (1 << TWINT) | (1 << TWSTO) |
(1 << TWEN);
}

static void i2c_write(uint8_t data)
{
    TWDR = data;
    TWCR = (1 << TWINT) | (1 << TWEN);
    while (!(TWCR & (1 << TWINT)));
}

#define OLED_W   0x78   /* (0x3C << 1) | 0
*/
#define SCR_W    128
#define SCR_PG   8

static void oled_cmd(uint8_t c)
{
    i2c_start();
    i2c_write(OLED_W);
    i2c_write(0x00);  /* command */
    i2c_write(c);
    i2c_stop();
}
static void oled_dat(uint8_t d)
{
    i2c_start();
    i2c_write(OLED_W);
    i2c_write(0x40);  /* data */
    i2c_write(d);
    i2c_stop();
}
static void oled_init(void)
{
    uint8_t i;
    static const uint8_t seq[] PROGMEM = {
        0xAE,
        0xD5, 0x80,
        0xA8, 0x3F,
        0xD3, 0x00,
        0x40,
        0x8D, 0x14,
        0x20, 0x00,
        0xA1,
        0xC8,
        0xDA, 0x12,
        0x81, 0xCF,
        0xD9, 0xF1,
        0xDB, 0x40,
        0xA4,
        0xA6,
        0xAF
    };
    _delay_ms(100);
    for (i = 0; i < sizeof(seq); i++)
        oled_cmd(pgm_read_byte(&seq[i]));
}

static void oled_pos(uint8_t page, uint8_t
col)
{
    oled_cmd((uint8_t)(0xB0 | (page &
0x07)));
    oled_cmd((uint8_t)(0x00 | (col  &
0x0F)));
    oled_cmd((uint8_t)(0x10 | (col  >>
4)));
}

static void oled_clear(void)
{
    uint8_t p;
    uint8_t c;
    for (p = 0; p < SCR_PG; p++) {
        oled_pos(p, 0);
        for (c = 0; c < SCR_W; c++)
oled_dat(0x00);
    }
}
```

```
static void oled_row(uint8_t page, uint8_t
pat)
{
    uint8_t c;
    oled_pos(page, 0);
    for (c = 0; c < SCR_W; c++)
oled_dat(pat);
}

static const uint8_t FONT[][5] PROGMEM = {
{0x00,0x00,0x00,0x00,0x00}, /* 0x20 ' ' */
    {0x00,0x00,0x5F,0x00,0x00}, /* '!' */
    {0x00,0x07,0x00,0x07,0x00}, /* '"' */
    {0x14,0x7F,0x14,0x7F,0x14}, /* '#' */
    {0x24,0x2A,0x7F,0x2A,0x12}, /* '$' */
    {0x23,0x13,0x08,0x64,0x62}, /* '%' */
    {0x36,0x49,0x55,0x22,0x50}, /* '&' */
    {0x00,0x05,0x03,0x00,0x00}, /* '\'' */
    {0x00,0x1C,0x22,0x41,0x00}, /* '(' */
    {0x00,0x41,0x22,0x1C,0x00}, /* ')' */
    {0x08,0x2A,0x1C,0x2A,0x08}, /* '*' */
    {0x08,0x08,0x3E,0x08,0x08}, /* '+' */
    {0x00,0x50,0x30,0x00,0x00}, /* ',' */
    {0x08,0x08,0x08,0x08,0x08}, /* '-' */
    {0x00,0x60,0x60,0x00,0x00}, /* '.' */
    {0x20,0x10,0x08,0x04,0x02}, /* '/' */
    {0x3E,0x51,0x49,0x45,0x3E}, /* '0' */
    {0x00,0x42,0x7F,0x40,0x00}, /* '1' */
    {0x42,0x61,0x51,0x49,0x46}, /* '2' */
    {0x21,0x41,0x45,0x4B,0x31}, /* '3' */
    {0x18,0x14,0x12,0x7F,0x10}, /* '4' */
    {0x27,0x45,0x45,0x45,0x39}, /* '5' */
    {0x3C,0x4A,0x49,0x49,0x30}, /* '6' */
    {0x01,0x71,0x09,0x05,0x03}, /* '7' */
    {0x36,0x49,0x49,0x49,0x36}, /* '8' */
    {0x06,0x49,0x49,0x29,0x1E}, /* '9' */
    {0x00,0x36,0x36,0x00,0x00}, /* ':' */
    {0x00,0x56,0x36,0x00,0x00}, /* ';' */
    {0x08,0x14,0x22,0x41,0x00}, /* '<' */
    {0x14,0x14,0x14,0x14,0x14}, /* '=' */
    {0x00,0x41,0x22,0x14,0x08}, /* '>' */
    {0x02,0x01,0x51,0x09,0x06}, /* '?' */
    {0x32,0x49,0x79,0x41,0x3E}, /* '@' */
    {0x7E,0x11,0x11,0x11,0x7E}, /* 'A' */
    {0x7F,0x49,0x49,0x49,0x36}, /* 'B' */
    {0x3E,0x41,0x41,0x41,0x22}, /* 'C' */
    {0x7F,0x41,0x41,0x22,0x1C}, /* 'D' */
    {0x7F,0x49,0x49,0x49,0x41}, /* 'E' */
    {0x7F,0x09,0x09,0x09,0x01}, /* 'F' */
    {0x3E,0x41,0x49,0x49,0x7A}, /* 'G' */
    {0x7F,0x08,0x08,0x08,0x7F}, /* 'H' */
    {0x00,0x41,0x7F,0x41,0x00}, /* 'I' */
    {0x20,0x40,0x41,0x3F,0x01}, /* 'J' */
    {0x7F,0x08,0x14,0x22,0x41}, /* 'K' */
    {0x7F,0x40,0x40,0x40,0x40}, /* 'L' */
    {0x7F,0x02,0x04,0x02,0x7F}, /* 'M' */
    {0x7F,0x04,0x08,0x10,0x7F}, /* 'N' */
    {0x3E,0x41,0x41,0x41,0x3E}, /* 'O' */
    {0x7F,0x09,0x09,0x09,0x06}, /* 'P' */
    {0x3E,0x41,0x51,0x21,0x5E}, /* 'Q' */
    {0x7F,0x09,0x19,0x29,0x46}, /* 'R' */
    {0x46,0x49,0x49,0x49,0x31}, /* 'S' */
    {0x01,0x01,0x7F,0x01,0x01}, /* 'T' */
    {0x3F,0x40,0x40,0x40,0x3F}, /* 'U' */
    {0x1F,0x20,0x40,0x20,0x1F}, /* 'V' */
    {0x3F,0x40,0x38,0x40,0x3F}, /* 'W' */
    {0x63,0x14,0x08,0x14,0x63}, /* 'X' */
    {0x07,0x08,0x70,0x08,0x07}, /* 'Y' */
    {0x61,0x51,0x49,0x45,0x43}, /* 'Z' */
    {0x00,0x7F,0x41,0x41,0x00}, /* '[' */
    {0x02,0x04,0x08,0x10,0x20}, /* '\\' */
    {0x00,0x41,0x41,0x7F,0x00}, /* ']' */
    {0x04,0x02,0x01,0x02,0x04}, /* '^' */
    {0x40,0x40,0x40,0x40,0x40}, /* '_' */
    {0x00,0x01,0x02,0x04,0x00}, /* '`' */
    {0x20,0x54,0x54,0x54,0x78}, /* 'a' */
    {0x7F,0x48,0x44,0x44,0x38}, /* 'b' */
    {0x38,0x44,0x44,0x44,0x20}, /* 'c' */
    {0x38,0x44,0x44,0x48,0x7F}, /* 'd' */
    {0x38,0x54,0x54,0x54,0x18}, /* 'e' */
    {0x08,0x7E,0x09,0x01,0x02}, /* 'f' */
    {0x0C,0x52,0x52,0x52,0x3E}, /* 'g' */
    {0x7F,0x08,0x04,0x04,0x78}, /* 'h' */
    {0x00,0x44,0x7D,0x40,0x00}, /* 'i' */
    {0x20,0x40,0x44,0x3D,0x00}, /* 'j' */
    {0x7F,0x10,0x28,0x44,0x00}, /* 'k' */
    {0x00,0x41,0x7F,0x40,0x00}, /* 'l' */
    {0x7C,0x04,0x18,0x04,0x78}, /* 'm' */
    {0x7C,0x08,0x04,0x04,0x78}, /* 'n' */
    {0x38,0x44,0x44,0x44,0x38}, /* 'o' */
    {0x7C,0x14,0x14,0x14,0x08}, /* 'p' */
    {0x08,0x14,0x14,0x18,0x7C}, /* 'q' */
    {0x7C,0x08,0x04,0x04,0x08}, /* 'r' */
    {0x48,0x54,0x54,0x54,0x20}, /* 's' */
    {0x04,0x3F,0x44,0x40,0x20}, /* 't' */
    {0x3C,0x40,0x40,0x20,0x7C}, /* 'u' */
    {0x1C,0x20,0x40,0x20,0x1C}, /* 'v' */
    {0x3C,0x40,0x30,0x40,0x3C}, /* 'w' */
    {0x44,0x28,0x10,0x28,0x44}, /* 'x' */
    {0x0C,0x50,0x50,0x50,0x3C}, /* 'y' */
    {0x44,0x64,0x54,0x4C,0x44}, /* 'z' */
};
/* Write one character */
static void oled_ch(uint8_t page, uint8_t
col, char c)
{
    uint8_t i;
    if (c < 0x20 || c > 0x7A) c = ' ';
    oled_pos(page, col);
    for (i = 0; i < 5; i++)
        oled_dat(pgm_read_byte(&FONT[c -
0x20][i]));
    oled_dat(0x00);
}
```

```
/* Write string — same as oled_str in
System 319 */
static void oled_str(uint8_t page, uint8_t
col, const char *s)
{
    while (*s && col < SCR_W) {
        oled_ch(page, col, *s++);
        col += 6;
    }
}

static void uart_init(void)
{
    UBRR0H = (uint8_t)(UBRR_VAL >> 8);
    UBRR0L = (uint8_t)(UBRR_VAL);
    UCSR0B = (1 << TXEN0);
    UCSR0C = (3 << UCSZ00);
}
static void uart_char(char c)
{
    while (!(UCSR0A & (1 << UDRE0)));
    UDR0 = (uint8_t)c;
}
static void uart_str(const char *s) {
while (*s) uart_char(*s++); }

static float adc_read(uint8_t ch)
{
    ADMUX  = (1 << REFS0) | (ch & 0x07);
    ADCSRA =
(1<<ADEN)|(1<<ADSC)|(1<<ADPS2)|(1<<ADPS1)|
(1<<ADPS0);
    while (ADCSRA & (1 << ADSC));
    return (float)ADC;
}

static uint8_t findMaxIndex(float arr[],
uint8_t size)
{
    int i;
    uint8_t idx = 0;
    for (i = 1; i < size; i++)
        if (arr[i] > arr[idx]) idx =
(uint8_t)i;
    return idx;
}
static uint8_t findMinIndex(float arr[],
uint8_t size)
{
    int i;
    uint8_t idx = 0;
    for (i = 1; i < size; i++)
        if (arr[i] < arr[idx]) idx =
(uint8_t)i;
    return idx;
}
```

```
static void oled_splash(void)
{
    oled_clear();
    oled_row(0, 0xFF);
    oled_str(1,  8, "** VOLTMETER **");
    oled_str(2, 14, "ZMPT101B Sensor");
    oled_row(3, 0xFF);
    oled_str(4,  4, "Lecturer:");
    oled_str(5,  4, "Sir Hassan-Ul-Haq");
    oled_str(6,  4, "Lead: M. Taha EE-
23319");
    oled_row(7, 0xFF);
}

static void oled_update(float vavg, float
vrms)
{
    char buf[12];
    char line[20];
    uint8_t k;

    /* Page 0 — header */
    oled_row(0, 0xFF);
    oled_str(0, 10, "=== VOLTMETER ===");

    /* Page 1 — solid line */
    oled_row(1, 0xFF);

    /* Page 2 — Vavg */
    oled_row(2, 0x00);
    dtostrf(vavg, 7, 3, buf);
    line[0]='V'; line[1]='a'; line[2]='v';
line[3]='g';
    line[4]=':'; line[5]=' ';
    for (k = 0; k < 7; k++) line[6+k] =
buf[k];
    line[13]=' '; line[14]='V';
line[15]='\0';
    oled_str(2, 4, line);

    /* Page 3 — blank */
    oled_row(3, 0x00);

    /* Page 4 — Vrms */
    oled_row(4, 0x00);
    dtostrf(vrms, 7, 3, buf);
    line[0]='V'; line[1]='r'; line[2]='m';
line[3]='s';
    line[4]=':'; line[5]=' ';
    for (k = 0; k < 7; k++) line[6+k] =
buf[k];
    line[13]=' '; line[14]='V';
line[15]='\0';
    oled_str(4, 4, line);

    /* Page 5 — blank */
```

```
    oled_row(5, 0x00);

    /* Page 6 — status */
    oled_row(6, 0x00);
    oled_str(6, 4, "AC 50Hz  ZMPT101B");

    /* Page 7 — solid line */
    oled_row(7, 0xFF);
}

int main(void)
{
    float   voltage[200];
    float   Vavg;
    float   Vrms;
    uint8_t max_idx;
    uint8_t min_idx;
    uint8_t half;
    uint8_t i;
    char    buf[12];

    uart_init();
    i2c_init();
    oled_init();
    oled_clear();

    /* Splash */
    oled_splash();

    /* Tera Term splash */
    uart_str("\r\n");

uart_str("========================\r\n");
    uart_str("        ** VOLTMETER **
\r\n");
    uart_str("     ZMPT101B AC Sensor
\r\n");
    uart_str("--------------------\r\n");
    uart_str(" Lecturer : Sir Hassan-Ul-
Haq     \r\n");
    uart_str(" Lead     : Muhammad Taha
EE-319  \r\n");

uart_str("========================\r\n");
    uart_str("\r\n");

    _delay_ms(2000);
    oled_clear();

    while (1)
    {
        /* 1. Sample */
        for (i = 0; i < 200; i++)
            voltage[i] = adc_read(0) *
(5.0f / 1023.0f);
        /* 2. Peaks */
        max_idx = findMaxIndex(voltage,
200);
        min_idx = findMinIndex(voltage,
200);
        /* 3. DC offset */
        Vavg = 0.0f;
        half = (max_idx > min_idx) ?
(max_idx - min_idx)
                                   :
(min_idx - max_idx);

        /* Safety: half must be 1-99 so
2*half stays within 200 */
        if (half < 1)   half = 1;
        if (half > 99)  half = 99;

        for (i = 0; i < (2 * half); i++)
Vavg += voltage[i];
        Vavg = (float)fabs(Vavg / (2.0f *
(float)half));

        /* 4. Remove offset */
        for (i = 0; i < 200; i++)
voltage[i] -= Vavg;

        /* 5. RMS */
        Vrms = 0.0f;
        for (i = 0; i < (2 * half); i++)
            Vrms += voltage[i] *
voltage[i];
        Vrms = (float)sqrt((double)(Vrms /
(2.0f * (float)half)));

        /* 6. Sanity check — INF/NaN guard
*/
        if (Vavg > 5.0f || Vavg < 0.0f)
Vavg = 0.0f;
        if (Vrms > 5.0f || Vrms < 0.0f)
Vrms = 0.0f;

        /* 7. UART — next line pe print */
        uart_str("  Vavg: ");
        dtostrf(Vavg, 6, 3, buf);
        uart_str(buf);
        uart_str(" V  |  Vrms: ");
        dtostrf(Vrms, 6, 3, buf);
        uart_str(buf);
        uart_str(" V\r\n");

        /* 7. OLED */
        oled_update(Vavg, Vrms);

        _delay_ms(500);
    }

    return 0;
}
```

RESULTS AND OBSERVATIONS:

The Arduino UNO successfully sampled the ZMPT101B AC voltage sensor output and computed the average (Vavg) and RMS (Vrms) voltages in real time. Upon startup, both the OLED display and Tera Term serial terminal displayed the project title, lecturer name, and group leader details. After the 2-second splash screen, live voltage readings began updating on the OLED and printing to Tera Term every 500ms.

The Tera Term serial terminal (9600 baud) displayed consistent and stable readings, confirming proper operation of the sensor, I2C OLED driver, and UART communication simultaneously. The oscilloscope verified the AC waveform at the ZMPT101B analog output, showing a sinusoidal signal at approximately 50 Hz.

From the Tera Term output, the following observations were recorded:

- **Average voltage (Vavg):** approximately 2.507 – 2.532 Volts confirming proper DC biasing at ADC mid-scale level (~2.5V), as expected from the ZMPT101B internal biasing circuit.
- **RMS voltage (Vrms):** approximately 0.343 – 0.349 Volts representing the stepped-down AC signal from the 220V mains at the sensor output.
- **OLED display:** simultaneously showed live Vavg and Vrms values, updating every 500ms with no flickering.
- **Readings stability:** consistent and repeatable across multiple sampling cycles, confirming reliable ADC sampling and calculation logic.

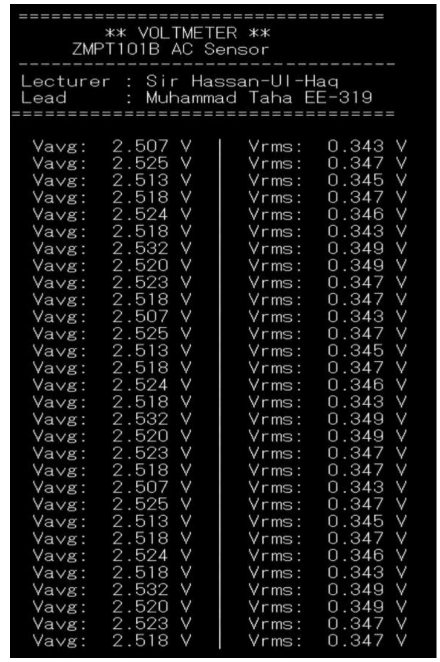

Figure 2: Tera Term Serial Monitor – Average and RMS Voltage Readings

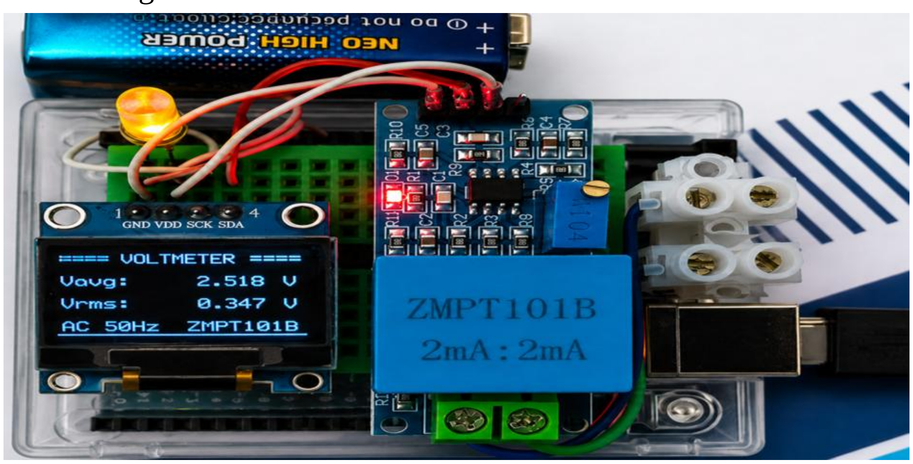

Figure 3: OLED Display – Average and RMS Voltage Readings

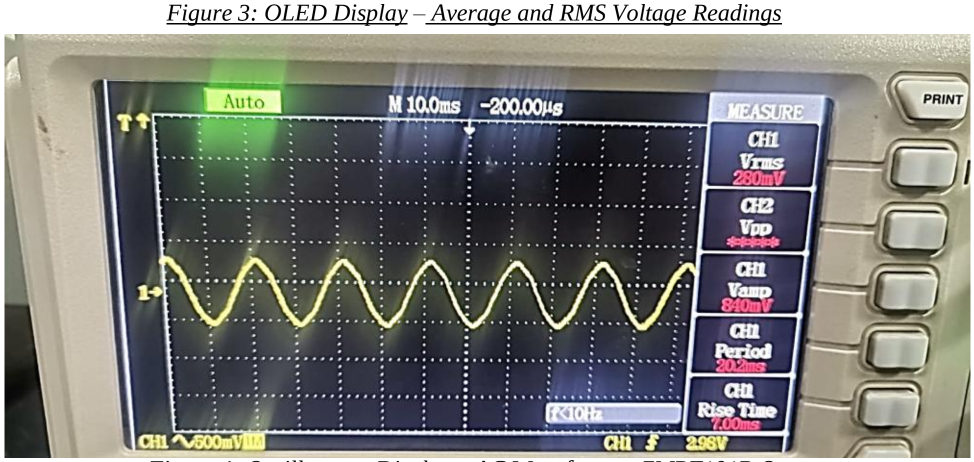

Figure 4: Oscilloscope Display – AC Waveform at ZMPT101B Output

Oscilloscope measurements on the ZMPT101B analog output revealed:

- CH1 Vrms = 280 mV
- CH1 Vamp (peak amplitude) = 840 mV
- Period = 20.2 ms → Frequency ≈ 49.5 Hz (consistent with 50 Hz mains)
- Rise Time = 7.00 ms
- The sinusoidal waveform is clean, confirming effective voltage transformation and signal conditioning.

DATA TABLE:

Reading #	Avg Voltage (V)	RMS Voltage (V)	Period (ms)	Frequency (Hz)
1	2.507	0.343	20.2	49.5
2	2.525	0.347	20.2	49.5
3	2.513	0.345	20.2	49.5
4	2.518	0.347	20.2	49.5
5	2.524	0.346	20.2	49.5
Average	2.517	0.346	20.2	49.5

CONCLUSION:

The lab successfully demonstrated the interfacing of the ZMPT101B analog AC voltage sensor with an Arduino UNO microcontroller for real-time AC voltage measurement. The system sampled 200 data points per measurement cycle and computed both average (Vavg) and RMS (Vrms) voltage values using custom AVR C code developed in Code::Blocks.

The results were displayed on two output interfaces simultaneously the SSD1306 OLED (128×64, I2C) and the Tera Term serial terminal (9600 baud) both functioning correctly at the same time, confirming stable I2C and UART communication on the ATmega328P. The OLED displayed a startup splash screen with project and team details, followed by continuously updating live voltage readings every 500ms.

The average voltage (Vavg) measured approximately 2.519V, confirming correct DC biasing at the ADC mid-scale level as expected from the ZMPT101B internal biasing circuit. The RMS voltage (Vrms) at the sensor output was approximately 0.346V, which when scaled by the appropriate calibration factor corresponds to the actual 220V AC mains voltage. The oscilloscope confirmed a clean sinusoidal waveform at approximately 50Hz, closely matching the Pakistan mains supply frequency.

The results validate the effectiveness of the ZMPT101B sensor for accurate, non-invasive AC voltage measurement in embedded systems. The project also demonstrated successful bare-metal AVR C programming without any external libraries, including a custom I2C driver, SSD1306 OLED driver, UART driver, and ADC sampling all written from scratch and flashed via AVRDUDE.

PROJECT SOURCE FILE:

https://github.com/Muhammad-Taha-Ansari/Digital-True-RMS-Voltmeter-using-ZMPT101B-and-Microcontroller